

```
#include <bits/stdc++.h> // Includes almost all standa

<iostream>           // cin, cout
<vector>             // vector
<string>             // string
<queue>             // queue, priority_queue
<stack>             // stack
<deque>             // deque
<list>             // list
<set>             // set, multiset
<map>             // map, multimap
<unordered_set>     // unordered_set
<unordered_map>     // unordered_map
<algorithm>         // sort, reverse, max, min, lower_boun
<numeric>           // accumulate, gcd, lcm, iota
<cmath>            // sqrt, pow, sin, cos, log
<climits>          // INT_MIN, INT_MAX
<limits>           // numeric_limits
<cstring>          // memset, memcpy, strlen
<cctype>           // isalpha, isdigit, toupper, tolower
<iomanip>          // setw, setprecision, fixed
<sstream>          // stringstream
```

```

<iomanip>           // setw, setprecision, fixed
<sstream>           // stringstream
<fstream>           // ifstream, ofstream, fstream
<functional>        // greater, less, function
<bitset>            // bitset
<tuple>             // tuple
<utility>           // pair, make_pair
<array>             // array
<chrono>            // time utilities
<random>            // random number generation
<regex>             // regular expressions
<iterator>          // iterators
<exception>         // exception handling
<typeinfo>          // typeid
    
```

Comparison

Method	Syntax	Advantage
<code>(*arr)[3]</code>	Pointer to array	Standard way
<code>arr[][3]</code>	Direct array	Sabse simple ✓
<code>**arr</code>	Double pointer	Dynamic arrays
Template	<code>(&arr)[r][c]</code>	Size auto detect ✓

Method 4: Template use karke (size bhi auto)

cpp



कॉपी करें

```
template<int row, int col>
void disp(int (&arr)[row][col]) { // ✅ size auto detect
    for(int i = 0; i < row; i++) {
        for(int j = 0; j < col; j++) {
            cout << arr[i][j] << "\t";
        }
        cout << endl;
    }
}
```

// Main mein:

```
int main() {
    int arr[][3] = {1,2,3,4,5,6,7,8,9};
    disp(arr); // row col dene ki zaroorat nahi!
}
```



Cpp Basic

Addition (A + B) — saari conditions

Condition	Possible?	Rule	Example	Result
Same row, same col $2 \times 2 + 2 \times 2$	Yes	Row(A)=Row(B) Col(A)=Col(B)	$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} + \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$	$\begin{bmatrix} 6 & 8 \\ 10 & 12 \end{bmatrix}$ 2×2
Same row, diff col $2 \times 2 + 2 \times 3$	No	Col must match	$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} + \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$	Error! Col 2 $\neq$ Col 3
Diff row, same col $2 \times 2 + 3 \times 2$	No	Row must match	$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} + \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}$	Error! Row 2 $\neq$ Row 3
Diff row, diff col $2 \times 2 + 3 \times 3$	No	Both must match	$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} + \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$	Error! Row & Col differ

Multiplication (A $\times$ B) — saari conditions

Condition	Possible?	Rule	Example	Result size
Same row, same col $2 \times 2 \times 2 \times 2$	Yes	Col(A)=Row(B) $2=2 \checkmark$	$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \times \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$	$\begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$ 2×2
Same row, diff col $2 \times 3 \times 3 \times 2$	Yes	Col(A)=Row(B) $3=3 \checkmark$	$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \times \begin{bmatrix} 7 & 8 \\ 9 & 10 \\ 11 & 12 \end{bmatrix}$	$\begin{bmatrix} 58 & 64 \\ 139 & 154 \end{bmatrix}$ $2 \times 2 =$ Row(A) $\times$ Col(B)
Diff row, same col $3 \times 2 \times 2 \times 3$	Yes	Col(A)=Row(B) $2=2 \checkmark$	$\begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix} \times \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$	$\begin{bmatrix} 9 & 12 & 15 \\ 19 & 26 & 33 \\ 29 & 40 & 51 \end{bmatrix}$ $3 \times 3 =$ Row(A) $\times$ Col(B)
Diff row, diff col $2 \times 3 \times 4 \times 2$	No	Col(A) $\neq$ Row(B) $3 \neq 4 \times$	$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \times \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \\ 7 & 8 \end{bmatrix}$	Error! Col(A)=3 $\neq$ Row(B)=4

Golden rules

Addition: $A(m \times n) + B(m \times n) \rightarrow \text{Result}(m \times n)$ | Dono bilkul same hone chahiye

Multiplication: $A(m \times n) \times B(n \times p) \rightarrow \text{Result}(m \times p)$ | A ka col = B ka row

Case 1 — use karo — modify karna ho:

```
cpp

void toUpper(string &str) { //  original badlega
    for(char &ch : str)
        ch = toupper(ch);
}

string s = "tarun";
toUpper(s);
cout << s; // TARUN — original badal gaya
```

Vector Table:

Feature	<code>vector<int> v</code>	<code>vector<int> &v</code>	<code>const vector<int> &v</code>
Kya pass hota hai	Copy	Original	Original (read-only)
Original badlega?	 Nahi	 Haan	 Nahi
Memory	Zyada (copy)	Kam	Kam
Modify kar sakte?	Copy pe	 Haan	 Nahi
Kab use karo	Recursion	Sort, modify	Sirf print/search

Table:

Feature	Value <code>string str</code>	Reference <code>string &str</code>
Kya pass hota hai	Copy	Original ka address
Original badlega?	❌ Nahi	✅ Haan
Memory	Zyada (copy banta hai)	Kam (copy nahi)
Speed	Slow (badi string ho toh)	Fast
Temporary pass kar sakte?	✅ Haan	❌ Nahi
<code>substr()</code> pass kar sakte?	✅ Haan	❌ Nahi
<code>""</code> pass kar sakte?	✅ Haan	❌ Nahi



Table:

Feature	<code>int arr[]</code> / <code>int *arr</code>	<code>const int arr[]</code> / <code>const int *arr</code>
Kya pass hota hai	Address (pointer)	Address (pointer)
Original badlega?	✅ Haan	❌ Nahi
Copy banta hai?	❌ Kabhi nahi	❌ Kabhi nahi
Modify kar sakte?	✅ Haan	❌ Nahi
Size milta hai?	❌ Alag <code>n</code> chahiye	❌ Alag <code>n</code> chahiye
Kab use karo	Sort, reverse, modify	Sirf padhna, print

```
Decimal : 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15  
Hex      : 0 1 2 3 4 5 6 7 8 9 A B C D E F
```

```
Decimal : 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31  
Hex      : 10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E 1F
```

```
Decimal : 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47  
Hex      : 20 21 22 23 24 25 26 27 28 29 2A 2B 2C 2D 2E 2F
```

```
Decimal : 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63  
Hex      : 30 31 32 33 34 35 36 37 38 39 3A 3B 3C 3D 3E 3F
```

Feature	■ Static Object	■ Dynamic Object
Syntax	<code>Student s;</code>	<code>Student *s = new Student();</code>
Memory Location	Stack Memory	Heap Memory
Memory Allocation	Compile / Run time (automatic)	Run time using <code>new</code>
Access Operator	<code>.</code> (dot)	<code>-></code> (arrow)
Memory Deallocation	Automatic	Manual using <code>delete</code>
Lifetime	Ends when scope ends	Exists until <code>delete</code> is called
Speed	Faster	Slightly slower
Memory Leak Risk	No	Yes — if <code>delete</code> not used
Constructor Call	Automatic	Automatic when <code>new</code> is used
Destructor Call	Automatic at scope end	When <code>delete</code> is called
Size Flexibility	Fixed	Can be decided at runtime
Use Case	Small, temporary objects	Large objects, dynamic data structures

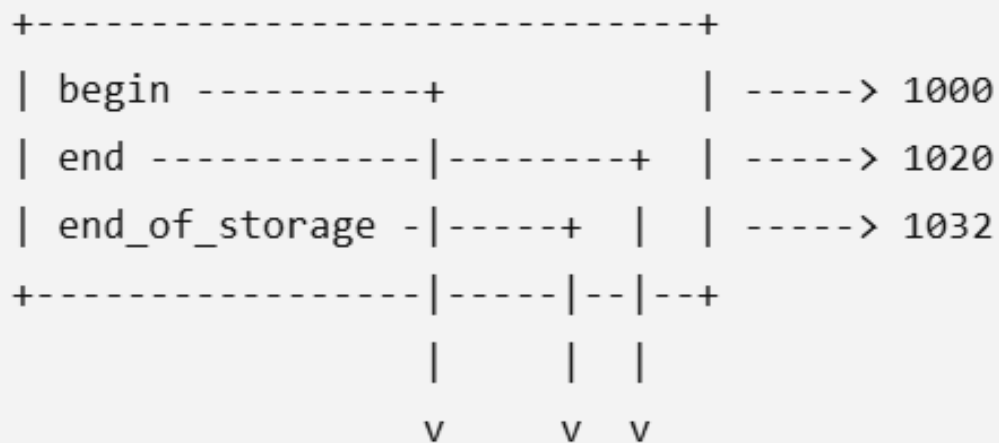
Array	Vector	Meaning	✦ Upg
<code>arr</code>	<code>v.data()</code>	पहले element का address	
<code>&arr[0]</code>	<code>&v[0]</code>	पहले element का address	
<code>&arr[1]</code>	<code>&v[1]</code>	दूसरे element का address	
<code>arr + 1</code>	<code>v.data() + 1</code>	दूसरे element का address	
<code>arr + 2</code>	<code>v.data() + 2</code>	तीसरे element का address	
<code>&arr</code>	<code>&v</code>	पूरे array / vector object का address	
<code>&arr + 1</code>	<code>&v + 1</code>	अगले array / vector object का address	
<code>*(arr)</code>	<code>*(v.data())</code> या <code>v[0]</code>	पहला element	
<code>*(arr + 1)</code>	<code>*(v.data() + 1)</code>	दूसरा element	



Expression	Address (32-bit Compiler)	✦ Upgrade
<code>v.data()</code>	<code>1000</code>	
<code>&v[0]</code>	<code>1000</code>	
<code>&v[1]</code>	<code>1004</code>	
<code>v.data()+1</code>	<code>1004</code>	
<code>&v</code>	<code>5000</code> (vector object का address)	
<code>&v+1</code>	<code>5012</code> (<code>5000 + sizeof(vector<int>)</code>), जहाँ <code>sizeof(vector<int>) = 12</code> bytes)	

```
vector<int> v = {10,20,30,40,50};
```

Stack (vector object)



Heap

Address	Value

1000	10
1004	20
1008	30
1012	40
1016	50
1020	unused
1024	unused
1028	unused
1032	end_of_storage



अब

```
begin          = 1000
end            = 1020
end_of_storage = 1032
```

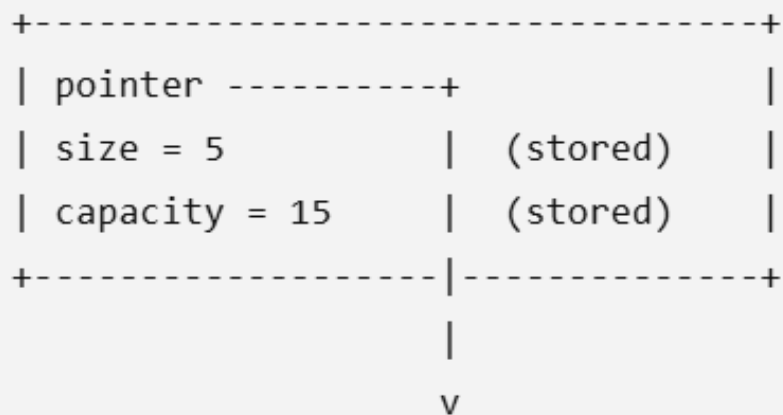
```
size      = (1020 - 1000) / 4 = 5
capacity = (1032 - 1000) / 4 = 8
```

Address Table

Expression	Meaning	Example Address
<code>str.data()</code>	पहले character का address	1000
<code>&str[0]</code>	पहले character का address	1000
<code>&str[1]</code>	दूसरे character का address	1001
<code>str.data()+1</code>	दूसरे character का address	1001
<code>str.c_str()</code>	C-style string का address	1000
<code>&str</code>	string object का address	5000
<code>&str+1</code>	अगले string object का address	5000 + sizeof(string)

```
string str = "hello";
```

Stack (string object)



Heap

```

1000 : 'h'
1001 : 'e'
1002 : 'l'
1003 : 'l'
1004 : 'o'
1005 : '\0'

```

Address table

Expression	Address	Meaning
<code>ch</code>	1000	पहले character का address
<code>&ch[0]</code>	1000	पहले character का address
<code>&ch[1]</code>	1001	दूसरे character का address
<code>ch+1</code>	1001	दूसरे character का address
<code>&ch</code>	1000	पूरे character array का address
<code>&ch+1</code>	1006	पूरे array के बाद का address ($6 \times \text{sizeof}(\text{char})$)

```
char ch[] = "hello";
```

Stack

```
1000 : 'h'
1001 : 'e'
1002 : 'l'
1003 : 'l'
1004 : 'o'
1005 : '\0'
```

```
p ×
pp > main()
    cout << "hello tarun" << endl;
    int arr[][3]={1,2,3,4,5,6,7,8,9};
    vector<vector<int>>v;
int row=sizeof(arr)/sizeof(arr[0]);
int col=sizeof(arr[0])/sizeof(arr[0][0]);

for(int i=0;i<row;i++){
    v.push_back(vector<int>(arr[i],arr[i]+col));
}
for(int i=0;i<row;i++){
    for(int j=0;j<col;j++){
        cout<<v[i][j]<<" ";
    }
    cout<<endl;
}
```

```
⟨/⟩ C++
```

```
vector<int>(arr[0], arr[0] + 2);
```

to sirf

```
1 2
```

copy hote.

Agar `arr[i] + 1` likhte

```
⟨/⟩ C++
```

```
vector<int>(arr[0], arr[0] + 1);
```

to sirf

```
1
```

```
vector<vector<int>> v = {  
    {1,2,3},  
    {4,5,6},  
    {7,8,9}  
};  
  
int row = v.size();  
int col = v[0].size();  
  
for(int i = 0; i < row; i++) {  
    v.push_back(vector<int>(v[i].begin(), v[i].end()));  
}  
  
for(auto &r : v) {  
    for(int x : r)  
        cout << x << " ";  
    cout << endl;  
}  
  
return 0;  
}
```



```
5  int main() {
6      int arr[][3]={1,2,3,4,5,6,7,8,9};
7
8      int row=sizeof(arr)/sizeof(arr[0]);
9      int col=sizeof(arr[0])/sizeof(arr[0][0]);
10     vector<vector<int>>v(row,vector<int>(col));
11
12     for(int i=0;i<row;i++){
13         for(int j=0;j<col;j++){
14             v[i][j]=arr[i][j];
15         }
16         for(auto &r : v) {
17             for(int x : r){
18                 cout << x << " ";
19             }
20             cout << endl;
21         }
22     }
```

Feature	Array (<code>int arr[]</code> / <code>int* arr</code>)	Array by Reference (<code>int (&arr)[N]</code>)	
Is the whole array copied?	✗ No	✗ No	
Original array can be modified?	✓ Yes	✓ Yes	
Parameter type	Pointer (<code>int*</code>)	Reference to array	
Array size known inside function?	✗ No (<code>sizeof(arr)</code> gives pointer size)	✓ Yes (<code>sizeof(arr)</code> gives full array size)	
Time Complexity of passing	<code>O(1)</code>	<code>O(1)</code>	
Extra Space	<code>O(1)</code>	<code>O(1)</code>	

2. Character Array (`char arr[]`)

Like normal arrays, character arrays also decay to pointers.

Feature	<code>char arr[]</code> / <code>char *arr</code>	<code>char (&arr)[N]</code>
Array copied?	✗ No	✗ No
Original modified?	✓ Yes	✓ Yes
Passing Time	<code>O(1)</code>	<code>O(1)</code>
Extra Space	<code>O(1)</code>	<code>O(1)</code>
Size known inside function?	✗ No	✓ Yes

Vector Pass by Value vs Pass by Reference

Feature	<code>vector<int> v</code> (Pass by Value)	<code>vector<int>& v</code> (Pass by Reference)
Vector copied?	✓ Yes	✗ No
Original vector modified?	✗ No	✓ Yes
Passing Time	$O(n)$	$O(1)$
Extra Space	$O(n)$	$O(1)$
Best for recursion?	✗ No	✓ Yes

1. string

Feature	<code>string s</code> (Pass by Value)	<code>string &s</code> / <code>const string &s</code> (Pass by Reference)
String copied?	✓ Yes	✗ No
Original string modified?	✗ No	✓ Yes (<code>string&</code>) / ✗ No (<code>const string&</code>)
Passing Time	$O(n)$	$O(1)$
Extra Space	$O(n)$	$O(1)$
Best for recursion	✗ No	✓ Yes

```
vector<int> v(arr, arr + 4);
```

Copies elements in the range:

```
[arr , arr+4)
```

Memory:

arr

Address

100 -> 1

104 -> 2

108 -> 3

112 -> 4

arr = 100

arr+4 = 116 (one past last)

Copied:

1 2 3 4



-b0C/ce1b2fa1

7. Iterator Constructor

```
</> C++  
  
vector<int> a = {10,20,30,40,50};  
  
vector<int> b(a.begin(), a.end());
```

Copies whole vector.

8. Partial Copy

```
</> C++  
  
vector<int> b(a.begin()+1, a.end()-1);
```

Output

20 30 40

Array → Vector

⌞ C++

```
int arr[] = {1,2,3,4};  
  
vector<int> v(arr, arr+4);
```

Vector → Array

⌞ C++

```
vector<int> v = {1,2,3};  
  
int arr[3];  
  
copy(v.begin(), v.end(), arr);
```

Vector → Vector

```
⟨/⟩ C++
```

```
vector<int> a={1,2,3};
```

```
vector<int> b(a.begin(),a.end());
```

or

```
⟨/⟩ C++
```

```
vector<int> b=a;
```

String → Vector<char>

⟨/⟩ C++

```
string s="hello";  
  
vector<char> v(s.begin(),s.end());
```

Output

h e l l o

Vector → String

⟨/⟩ C++

```
vector<char> v={'h','e','l','l','o'};  
  
string s(v.begin(),v.end());
```

first → Included ✓
last → Excluded ✕

Examples

⌕ C++

```
sort(v.begin(), v.end());

reverse(v.begin(), v.end());

find(v.begin(), v.end(), 10);

copy(v.begin(), v.end(), arr);

vector<int> b(v.begin(), v.end());
```

(C) Copy Partial Vector

```
</> C++
```

```
vector<int> a={1,2,3,4,5,6};  
  
vector<int> left(a.begin(),a.begin()+3);  
  
vector<int> right(a.begin()+3,a.end());
```

Output

```
left : 1 2 3  
right: 4 5 6
```

Divide Vector into Two Parts (Best Method)

Instead of loops,

```
</> C++  
  
int mid=v.size()/2;  
  
vector<int> left(v.begin(),v.begin()+mid);  
  
vector<int> right(v.begin()+mid,v.end());
```

No loops required.

Divide Array into Two Parts

```
</> C++  
  
int arr[]={1,2,3,4,5,6};  
  
int mid=6/2;  
  
vector<int> left(arr,arr+mid);  
  
vector<int> right(arr+mid,arr+6);
```

e-60c7ce1b2fa1

Merge Sort Example

Instead of

⟨/⟩ C++

```
vector<int> a(n1),b(n2);
```

```
for(int i=0;i<n1;i++)  
    a[i]=v[i];
```

```
for(int i=0;i<n2;i++)  
    b[i]=v[n1+i];
```

Write

⟨/⟩ C++

```
int mid=v.size()/2;
```

```
vector<int> a(v.begin(),v.begin()+mid);
```

```
vector<int> b(v.begin()+mid,v.end());
```



STL Functions for Copy

</> C++

```
copy(arr, arr+n, v.begin());
```

```
copy(v.begin(), v.end(), arr);
```

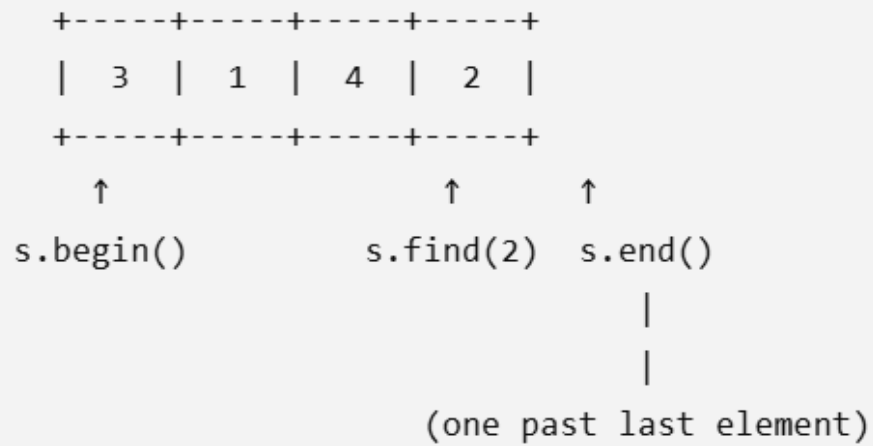
```
copy(v.begin(), v.end(), another.begin());
```

Upg

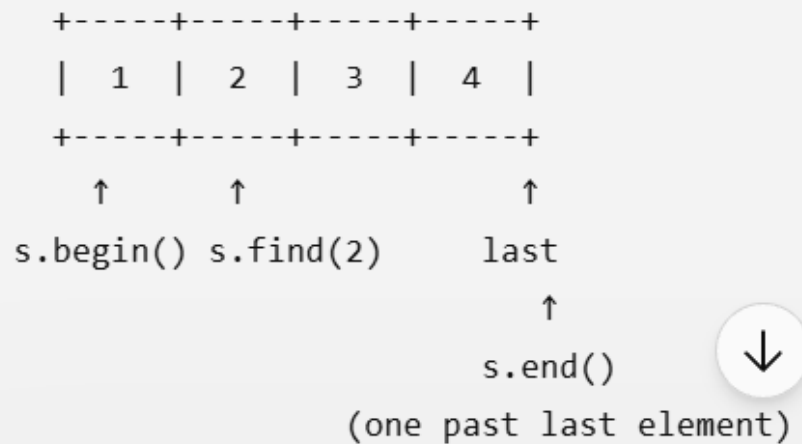
From	To	Syntax
Array	Array	for loop / copy()
Array	Vector	vector<int> v(arr, arr+n);
Vector	Array	copy(v.begin(), v.end(), arr);
Vector	Vector	vector<int> b=a;
Vector	Vector (Range)	vector<int> b(a.begin(), a.end());
Partial Vector	Vector	vector<int> b(a.begin()+l, a.begin()+r);
String	Vector<char>	vector<char> v(s.begin(), s.end());
Vector<char>	String	string s(v.begin(), v.end());



```
unordered_set<int> s
(order is NOT guaranteed)
```



```
set<int> s
(elements are sorted)
```



```
</> C++
```

```
set<char> st;
```

```
st.insert('D');
```

```
st.insert('B');
```

```
st.insert('F');
```

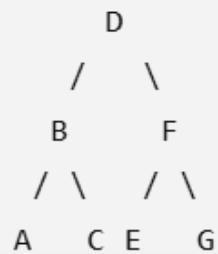
```
st.insert('A');
```

```
st.insert('C');
```

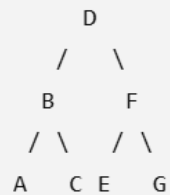
```
st.insert('E');
```

```
st.insert('G');
```

Conceptually memory me tree kuch aisa ho sakta hai:



Har node memory me alag-alag jagah hota hai.



Har node memory me alag-alag jagah hota hai.

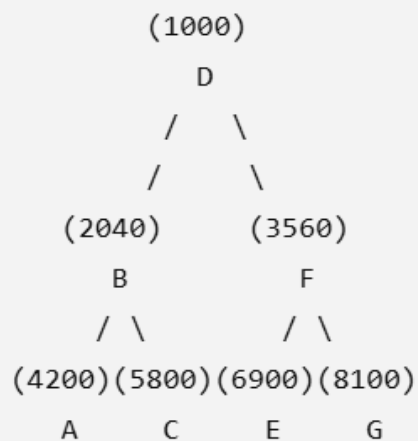
Maan lo memory addresses ye hain (sirf example):

Address	Data

1000	D
2040	B
3560	F
4200	A
5800	C
6900	E
8100	G

Har node me sirf value nahi hoti, balki pointers bhi hote hain.

Tree ko pointers connect karte hain.



`unordered_map<string,int> mp`

[Upgrade](#)

`</> C++`



```
mp["raghav ji"] = 33;
mp["tarun patidar"] = 3;
mp["shivam patidar"] = 4;
mp["amit"] = 25;
```

Maan lo `bucket_count = 10`.

Key	Example Hash Value	Bucket = Hash % 10
"raghav ji"	45673	45673 % 10 = 3
"tarun patidar"	98716	98716 % 10 = 6
"shivam patidar"	12345	12345 % 10 = 5
"amit"	76583	76583 % 10 = 3



Ab `unordered_map` ke buckets kuch is tarah dikh sakte hain:

```
unordered_map<string,int>
bucket_count = 10
```

```
Bucket 0 :
Bucket 1 :
Bucket 2 :
Bucket 3 : ("raghav ji",33) -> ("amit",25)
Bucket 4 :
Bucket 5 : ("shivam patidar",4)
Bucket 6 : ("tarun patidar",3)
Bucket 7 :
Bucket 8 :
Bucket 9 :
```

```
</> C++
```

```
map<string,int> mp;
```

```
mp["D"] = 40;
```

```
mp["B"] = 20;
```

```
mp["F"] = 60;
```

```
mp["A"] = 10;
```

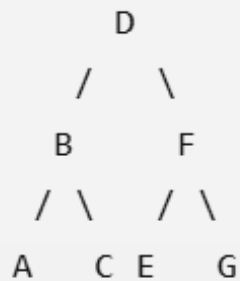
```
mp["C"] = 30;
```

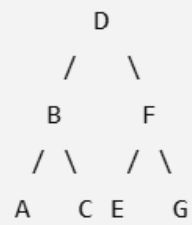
```
mp["E"] = 50;
```

```
mp["G"] = 70;
```

`map` internally **Red-Black Tree** use karta hai.

Conceptually tree kuch aisa socho:





Traversal

Output

Inorder (LNR)

A B C D E F G

Preorder (NLR)

D B A C F E G

Postorder (LRN)

A C B E G F D